



Computer System Overview

Chapter 1

Operating System

- Exploits the hardware resources of one or more processors to provide a set of services to system users
- Manages secondary memory and I/O devices



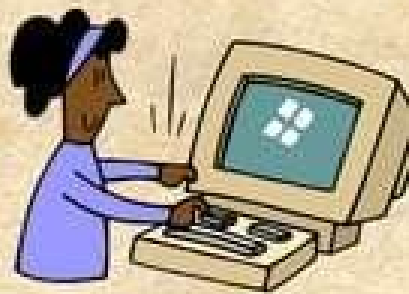
Basic Elements

Processor

**I/O
Modules**

**Main
Memory**

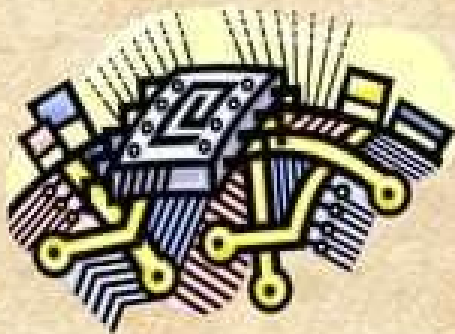
**System
Bus**



Processor

Controls the
operation of the
computer

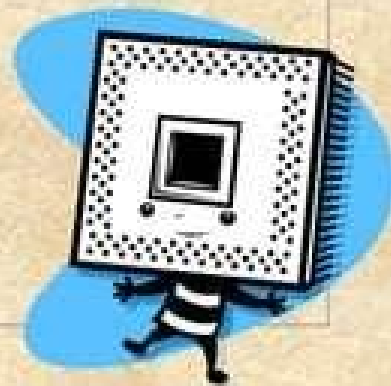
Performs the
data processing
functions



Referred to as
the *Central
Processing Unit*
(CPU)

Main Memory

- Volatile
- Contents of the memory is lost when the computer is shut down
- Referred to as real memory or primary memory



I/O Modules

Moves data
between the
computer and
external
environments
such as:

storage (e.g. hard
drive)

communications
equipment

terminals

System Bus



- Provides for communication among processors, main memory, and I/O modules

Top-Level Components

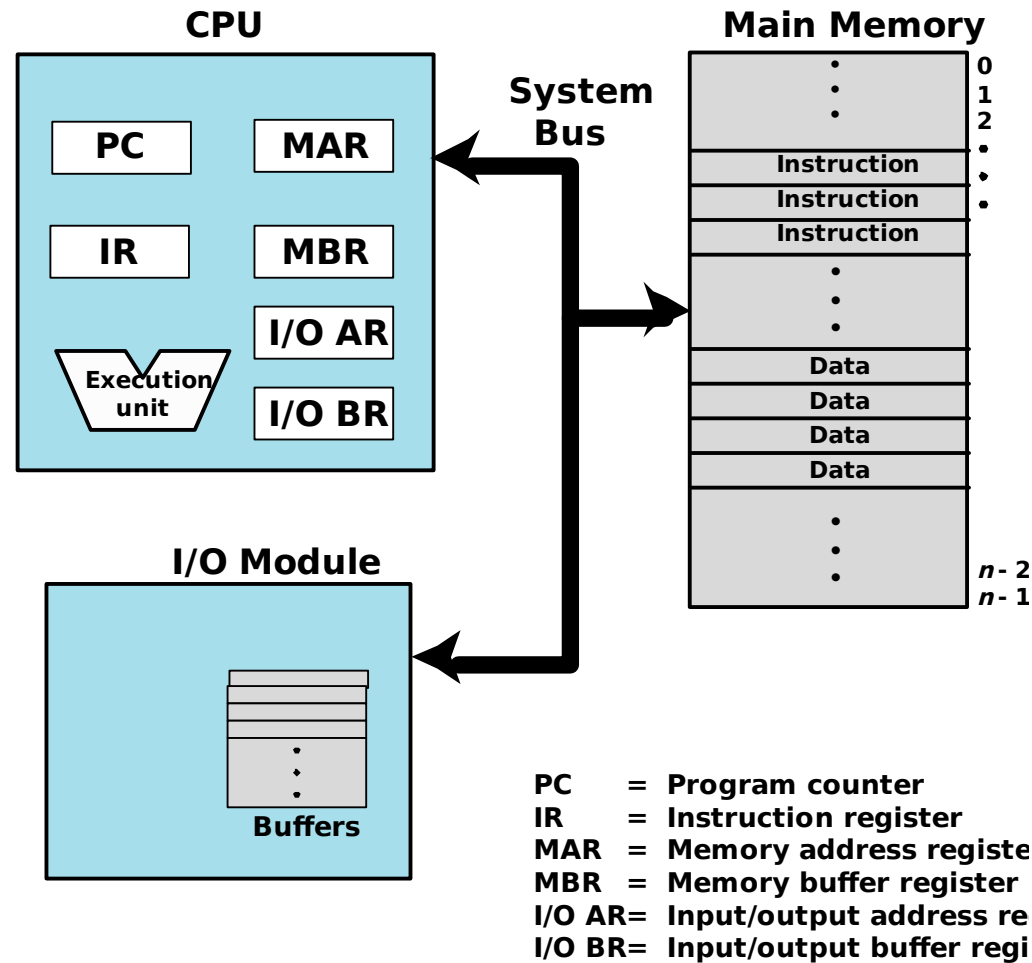


Figure 1.1 Computer Components: Top-Level \

Processor

- Two internal registers
 - Memory address register (MAR)
 - Specifies the address for the next read or write
 - Memory buffer register (MBR)
 - Contains data written into memory or receives data read from memory
 - I/O address register
 - I/O buffer register

Processor Registers

- User-visible registers
 - Enable programmer to minimize main-memory references by optimizing register use
- Control and status registers
 - Used by processor to control operating of the processor
 - Used by privileged operating-system routines to control the execution of programs

User-Visible Registers

- May be referenced by machine language
- Available to all programs - application programs and system programs
- Types of registers
 - Data
 - Address
 - Index
 - Segment pointer
 - Stack pointer

User-Visible Registers

- Address Registers
 - Index
 - Involves adding an index to a base value to get an address
 - Segment pointer
 - When memory is divided into segments, memory is referenced by a segment and an offset
 - Stack pointer
 - Points to top of stack

Control and Status Registers

- Program Counter (PC)
 - Contains the address of an instruction to be fetched
- Instruction Register (IR)
 - Contains the instruction most recently fetched
- Program Status Word (PSW)
 - Condition codes
 - Interrupt enable/disable
 - Supervisor/user mode

Control and Status Registers

- Condition Codes or Flags
 - Bits set by the processor hardware as a result of operations
 - Examples
 - Positive result
 - Negative result
 - Zero
 - Overflow

EVOLUTION OF THE MICROPROCESSOR

- Earlier : Single chip processor
- **Microprocessor**: Contained a processor on a single chip.
- They are now multiprocessors; each chip (**called a socket**) contains multiple processors (**called cores**), each with multiple levels of large memory caches, and multiple logical processors sharing the execution units of each core.
- **Graphical Processing Units (GPUs)** provide efficient computation on arrays of data using “Single-Instruction Multiple Data (SIMD)” techniques pioneered in supercomputers.

EVOLUTION OF THE MICROPROCESSOR

- Digital Signal Processors (DSPs) are also present, for dealing with streaming signals—such as audio or video.
- DSPs used to be embedded in I/O devices, like modems, but now becoming first-class computational devices, especially in handhelds
- **Summary:**
- To satisfy the requirements of handheld devices, instead of classic microprocessor “System on a Chip (SoC), CPUs and caches are on the same chip + such as DSPs, GPUs, I/O devices (such as radios and codecs), and main memory on single chip.

Instruction Execution

- Two steps
 - Processor reads instructions from memory
 - Fetches
 - Processor executes each instruction

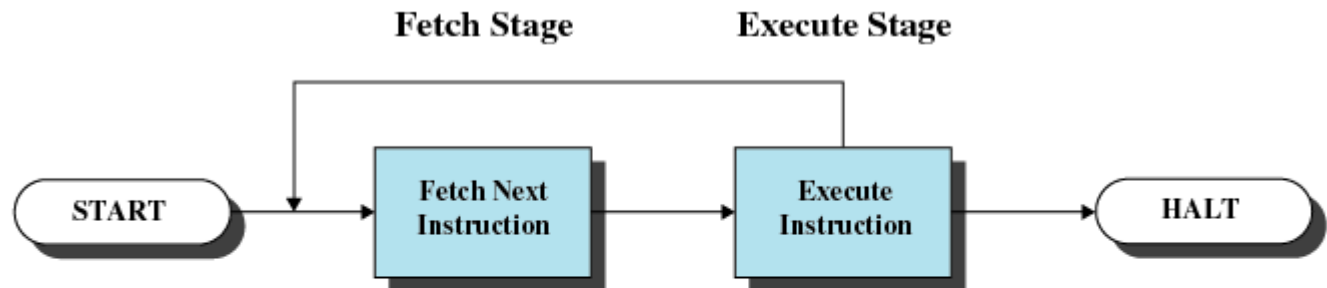


Figure 1.2 Basic Instruction Cycle

Instruction Fetch and Execute

- The processor fetches the instruction from memory
- Program counter (PC) holds address of the instruction to be fetched next
- Program counter is incremented after each fetch

Instruction Register

- Fetched instruction is placed in the instruction register.
- The instruction contains bits that specify the action the processor is to take.
- The processor interprets the instruction and performs the required action .
- These actions generally grouped into four categories.

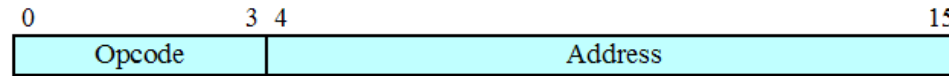
Instruction Register

Categories

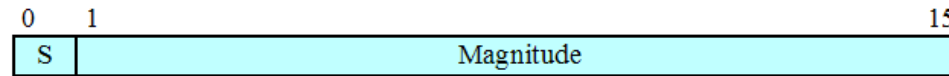
- Processor-memory
 - Transfer data between processor and memory
- Processor-I/O
 - Data transferred to or from a peripheral device
- Data processing
 - Arithmetic or logic operation on data
- Control
 - Alter sequence of execution

Characteristics of a Hypothetical Machine

Both instructions and data are 16 bits long, and memory is organized as a sequence of 16-bit words.



(a) Instruction format



(b) Integer format

Program Counter (PC) = Address of instruction
Instruction Register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

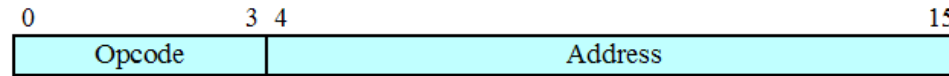
0001 = Load AC from Memory
0010 = Store AC to Memory
0101 = Add to AC from Memory

(d) Partial list of opcodes

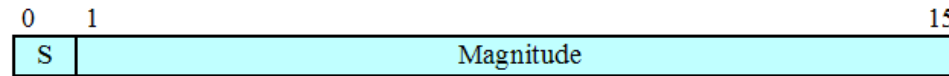
Figure 1.3 Characteristics of a Hypothetical Machine

Characteristics of a Hypothetical Machine

Both instructions and data are 16 bits long, and memory is organized as a sequence of 16-bit words.



(a) Instruction format



(b) Integer format

Program Counter (PC) = Address of instruction
Instruction Register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from Memory
0010 = Store AC to Memory
0101 = Add to AC from Memory

(d) Partial list of opcodes

Figure 1.3 Characteristics of a Hypothetical Machine

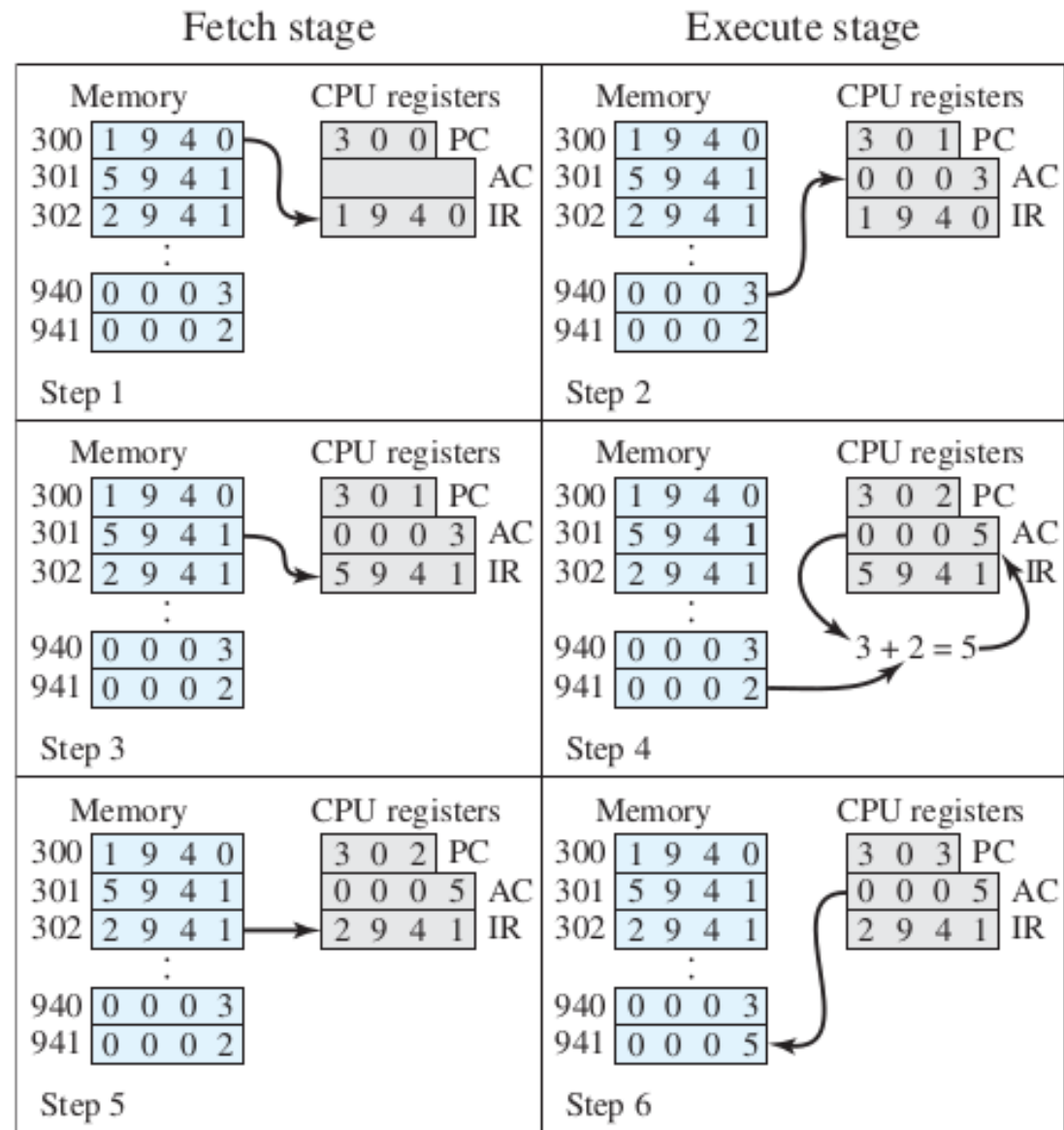


Figure 1.4 Example of Program Execution (contents of memory and registers in hexadecimal)

- The instruction format provides 4 bits for the opcode, allowing as many as 16 different opcodes (represented by a single hexadecimal 1 digit).
- The opcode defines the operation the processor is to perform. With the remaining 12 bits of the instruction format, up to 2^{12} 4,096 (4K) words of memory (denoted by three hexadecimal digits) can be directly addressed.

- 1) The PC contains 300, the address of the first instruction. This instruction (the value 1940 in hexadecimal) is loaded into the IR and the PC is incremented.

Note that this process involves the use of a memory address register (MAR) and a memory buffer register (MBR).

- 1) The first 4 bits (first hexadecimal digit) in the IR indicate that the AC is to be loaded from memory. The remaining 12 bits (three hexadecimal digits) specify the address, which is 940.
- 2) The next instruction (5941) is fetched from location 301 and the PC is incremented.
- 3) The old contents of the AC and the contents of location 941 are added and the result is stored in the AC.
- 4) The next instruction (2941) is fetched from location 302 and the PC is incremented.
- 5) The contents of the AC are stored in location 941.

Interrupts

A mechanism by which other modules (I/O, memory) may interrupt the normal sequencing of the processor.

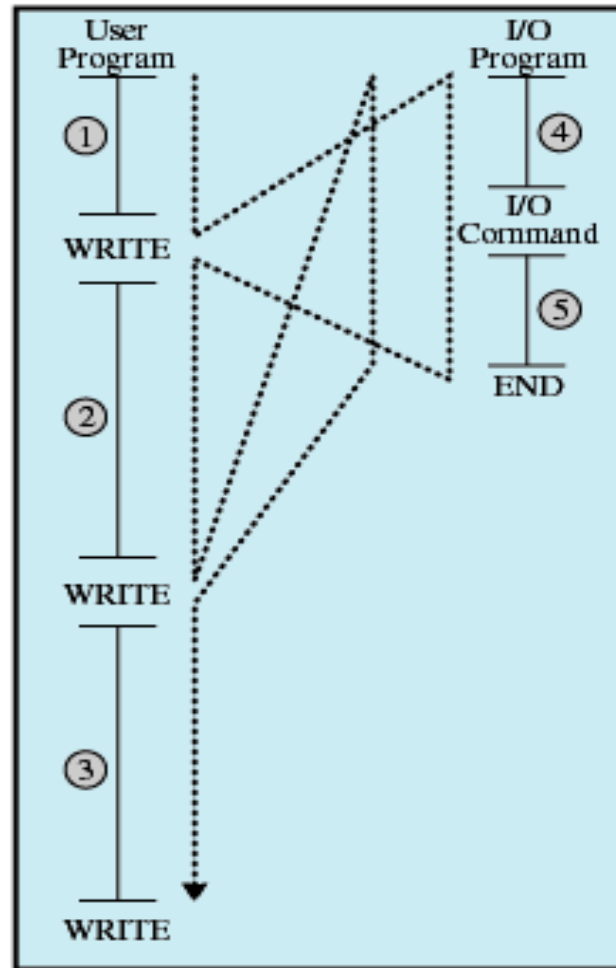
Table 1.1 Classes of Interrupts

Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, and reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation or to signal a variety of error conditions.
Hardware failure	Generated by a failure, such as power failure or memory parity error.

Interrupts

- Interrupts are provided primarily as a way to improve processor utilization.
- Most I/O devices are slower than the processor
 - Processor must pause to wait for device
 - E.g processor-printer

Program Flow of Control Without Interrupts



(a) No interrupts

Program Flow of Control Without Interrupts

- **A sequence of instructions, to prepare for the actual I/O operation.**

i.e This may include copying the data to be output into a special buffer and preparing the parameters for a device command.

- **The actual I/O command.**

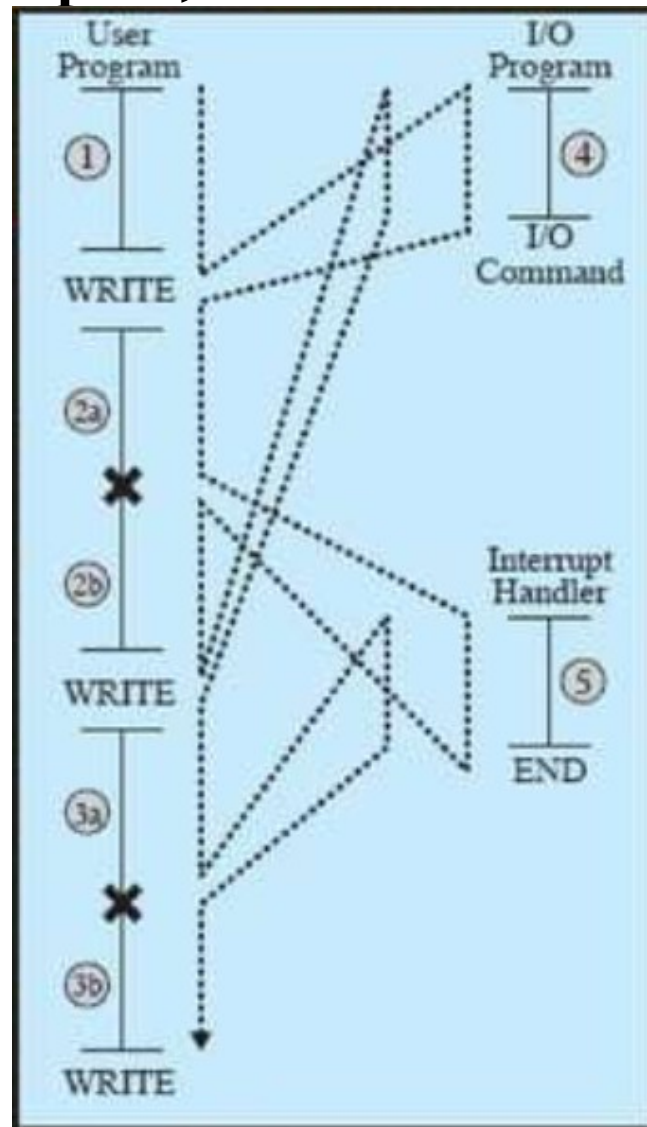
Without the use of interrupts, once this command is issued, the program must wait for the I/O device to perform the requested function (or periodically check the status, or poll, the I/O device).

The program might wait by simply repeatedly performing a test operation to determine if the I/O operation is done.

- **A sequence of instructions, to complete the operation.**

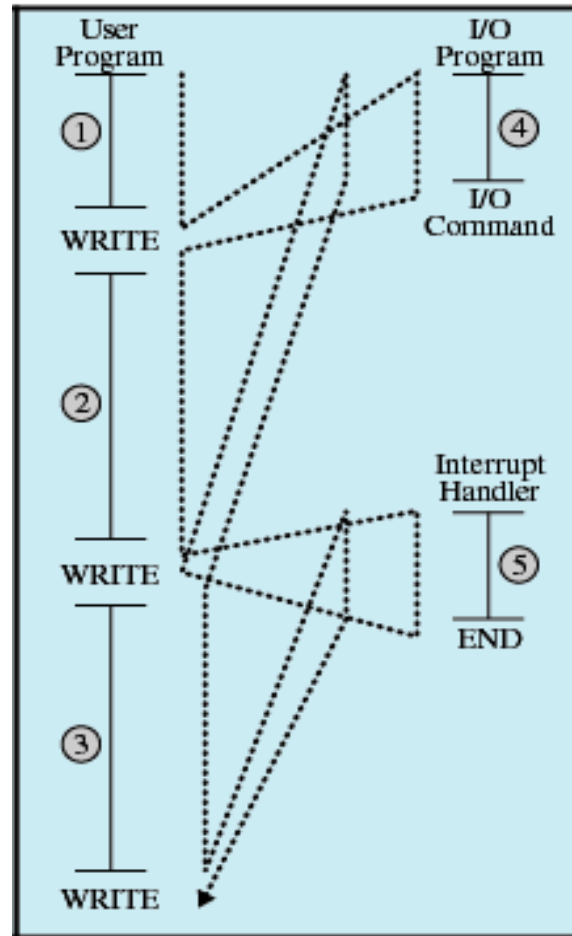
This may include setting a flag indicating the success or failure of the operation.

Program Flow of Control With Interrupts, Short I/O Wait



(b) Interrupts; short I/O wait

Program Flow of Control With Interrupts; Long I/O Wait



(c) Interrupts; long I/O wait

Interrupt Handler

- Program to service a particular I/O device
- Generally part of the operating system

Interrupts

- Suspends the normal sequence of execution

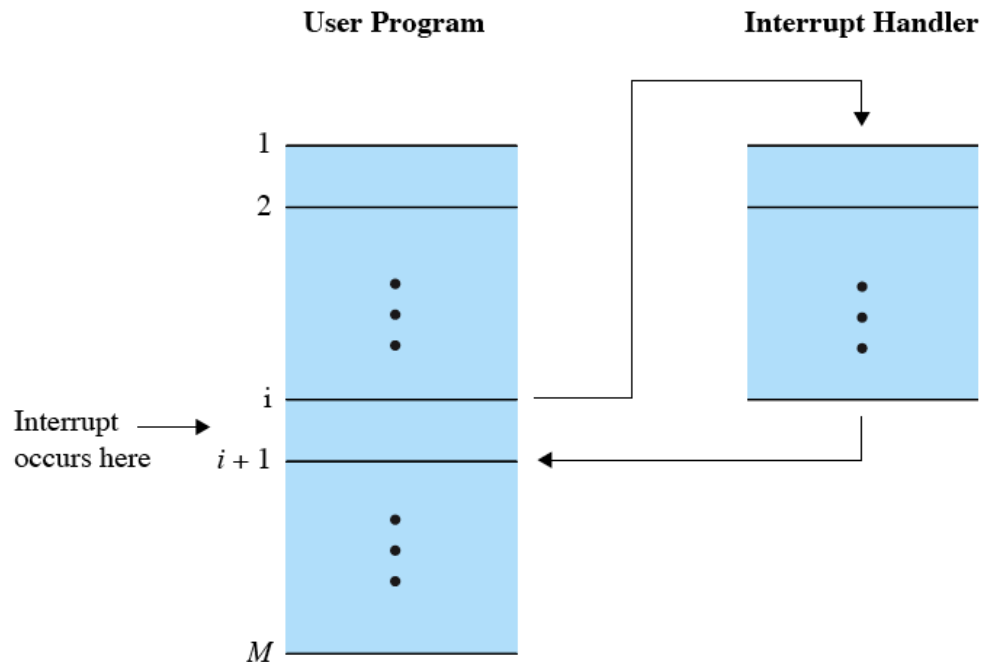


Figure 1.6 Transfer of Control via Interrupts

Interrupt Cycle

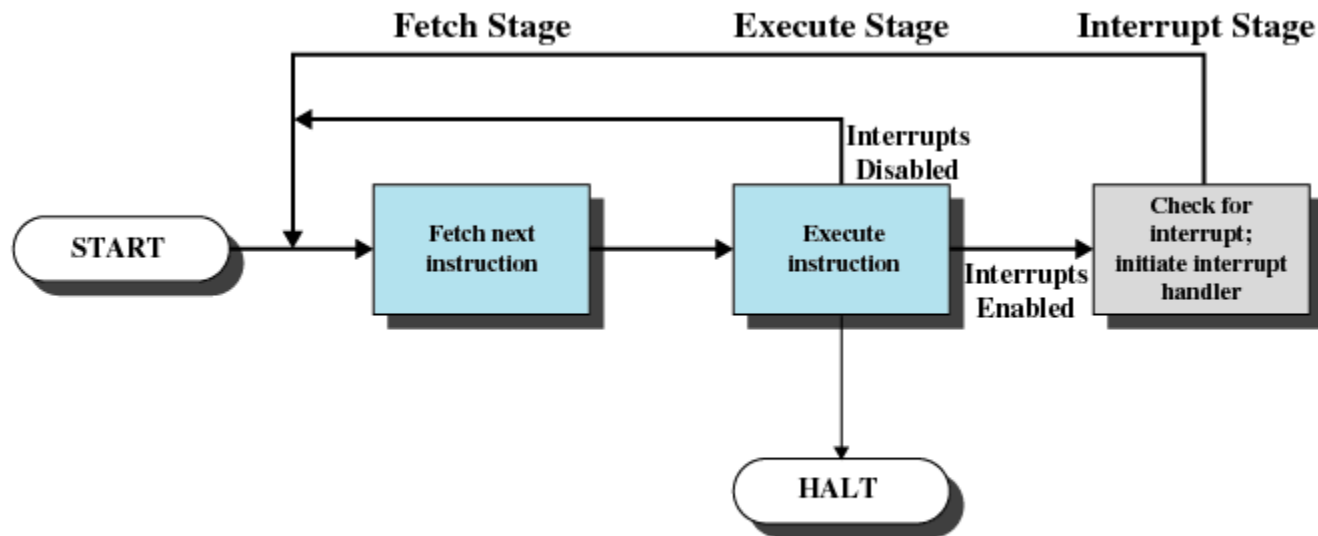


Figure 1.7 Instruction Cycle with Interrupts

Interrupt Cycle

- Processor checks for interrupts
- If no interrupts fetch the next instruction for the current program
- If an interrupt is pending, suspend execution of the current program, and execute the interrupt-handler routine

Timing Diagram Based on Short I/O Wait

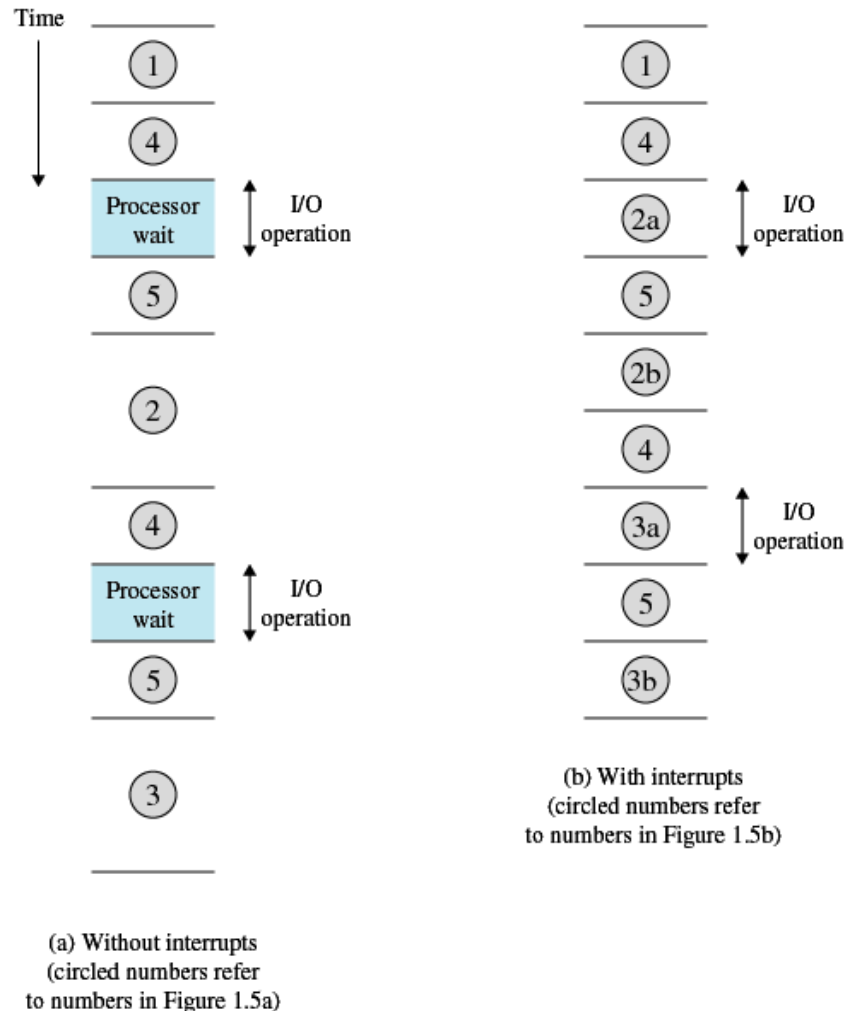


Figure 1.8 Program Timing: Short I/O Wait

Timing Diagram Based on long I/O Wait

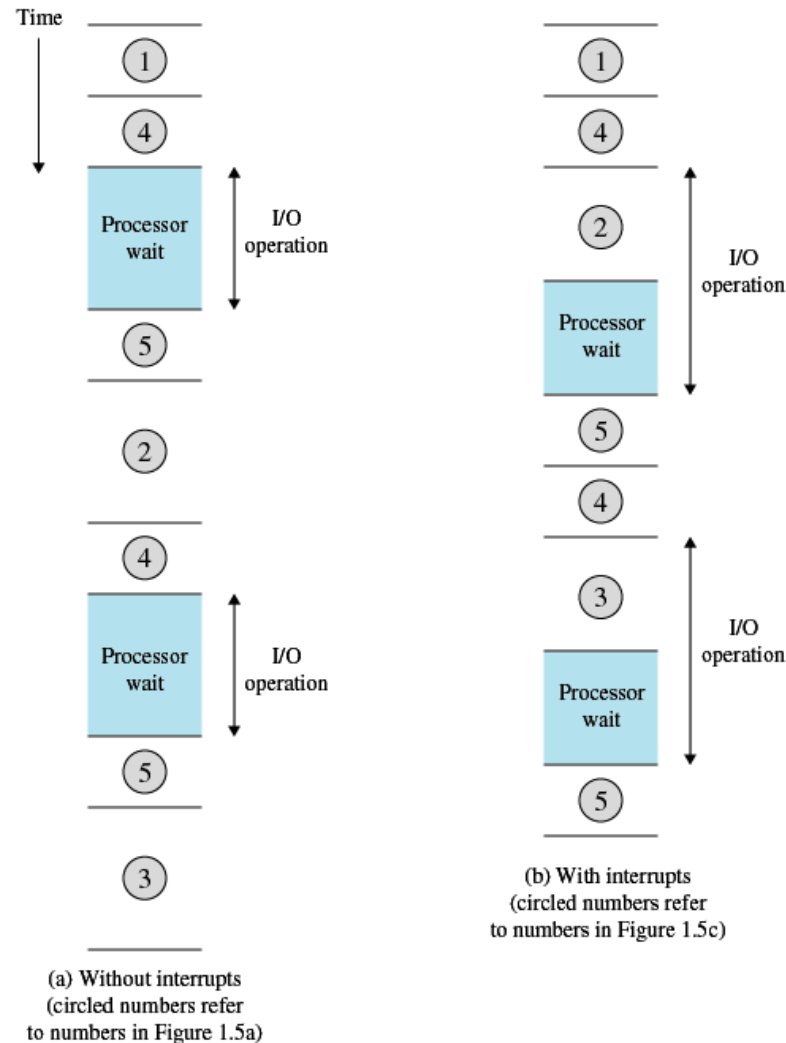
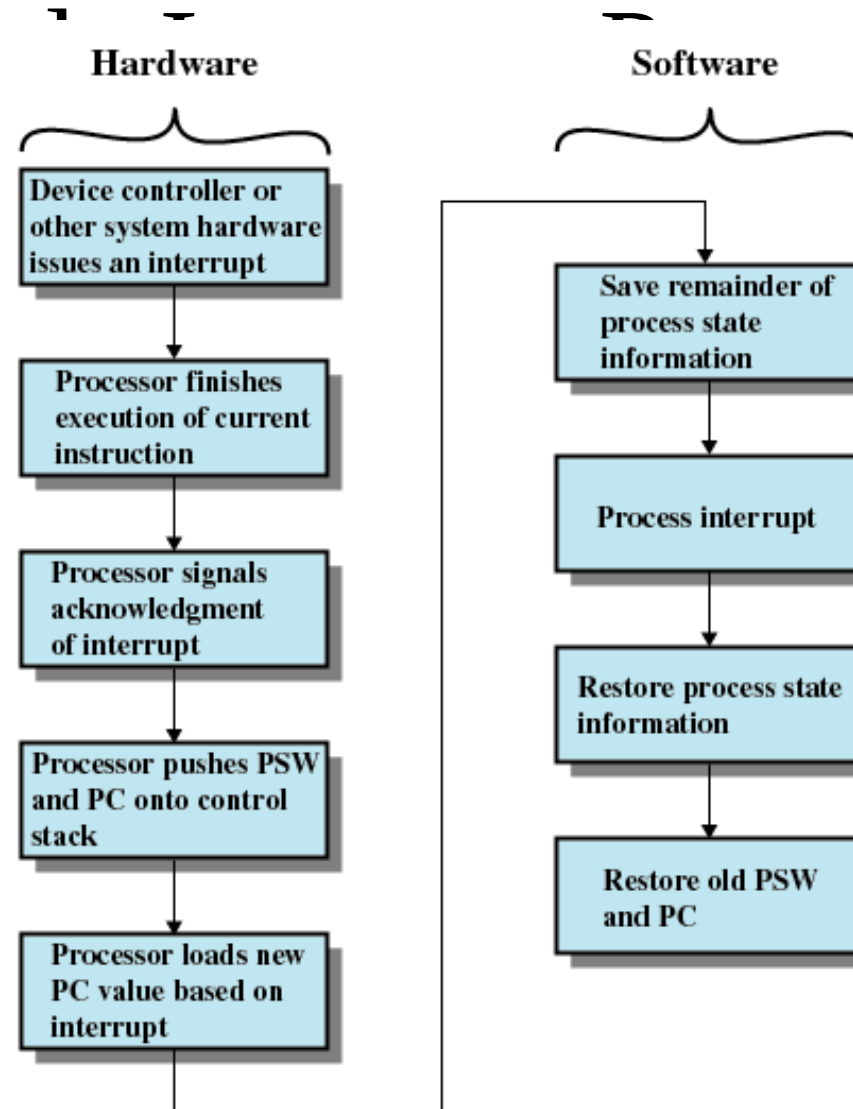


Figure 1.9 Program Timing: Long I/O Wait

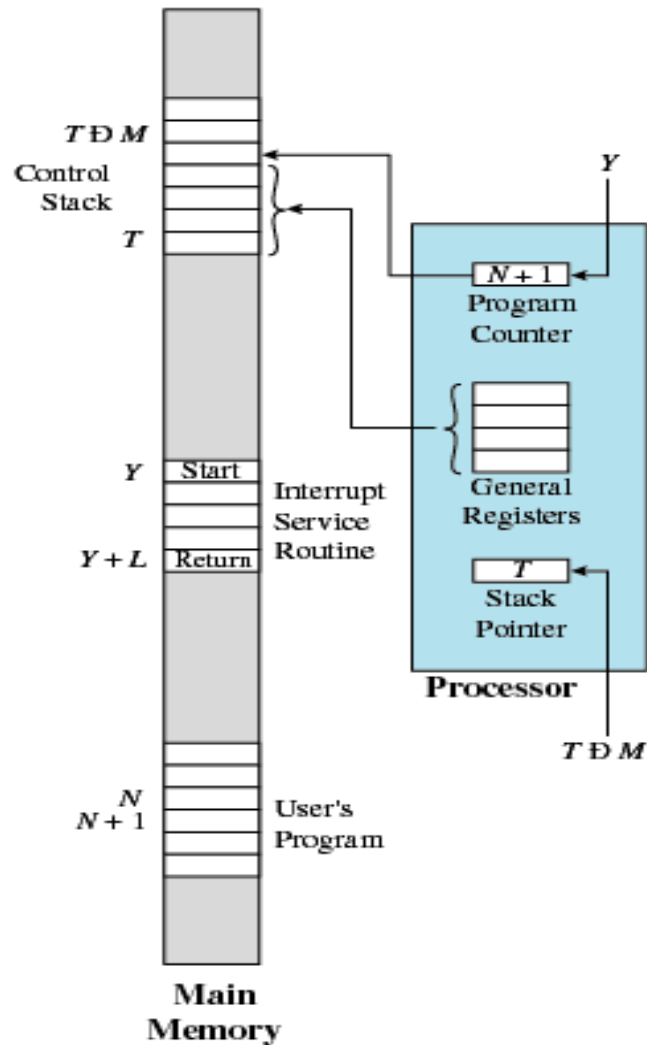
Sim



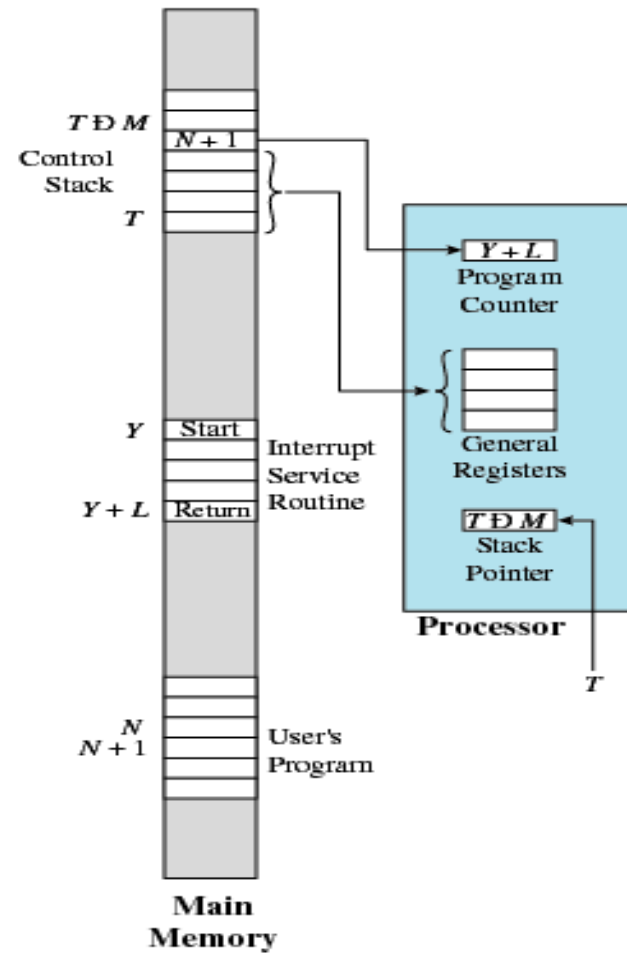
ing

Figure 1.10 Simple Interrupt Processing

Changes in Memory and Registers for an Interrupt



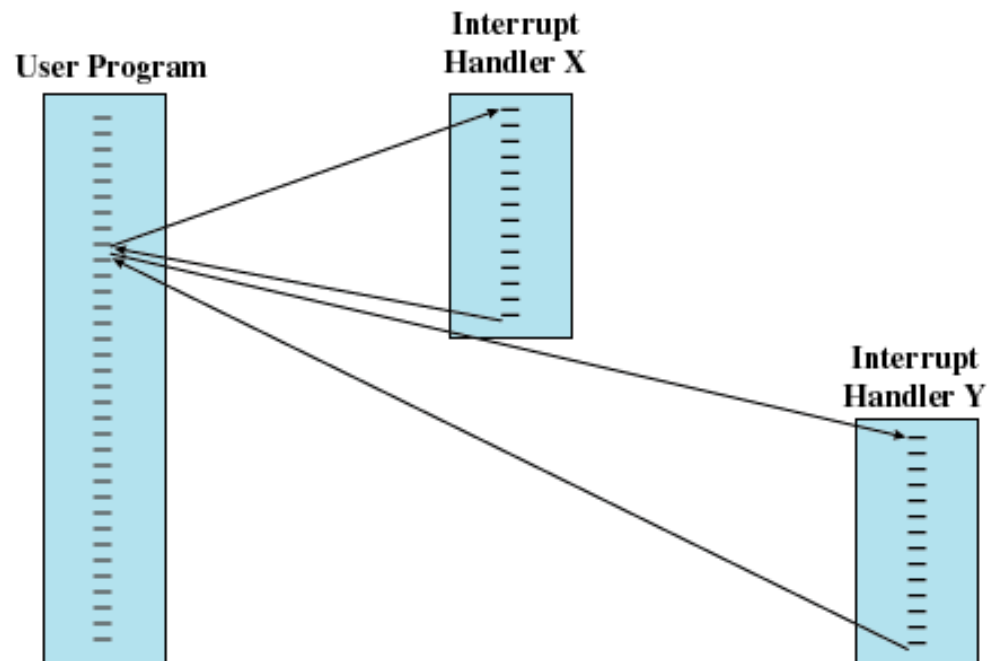
(a) Interrupt occurs after instruction at location N



(b) Return from interrupt

Multiple Interrupts

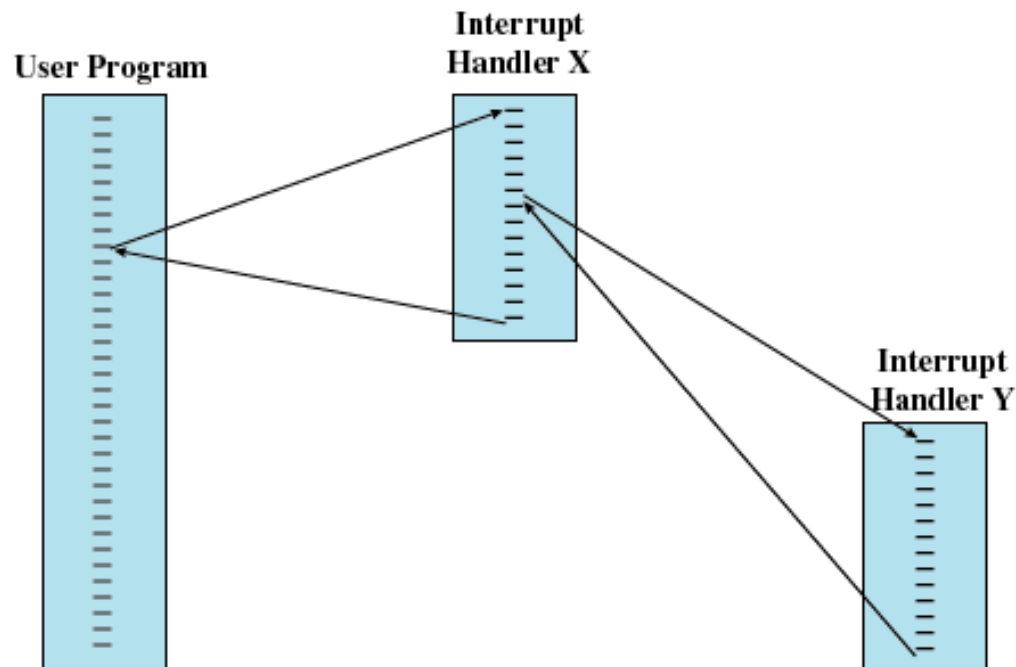
Disable interrupts while an interrupt is being processed



(a) Sequential interrupt processing

Multiple Interrupts

Define priorities for interrupts



(b) Nested interrupt processing

Multiple Interrupts

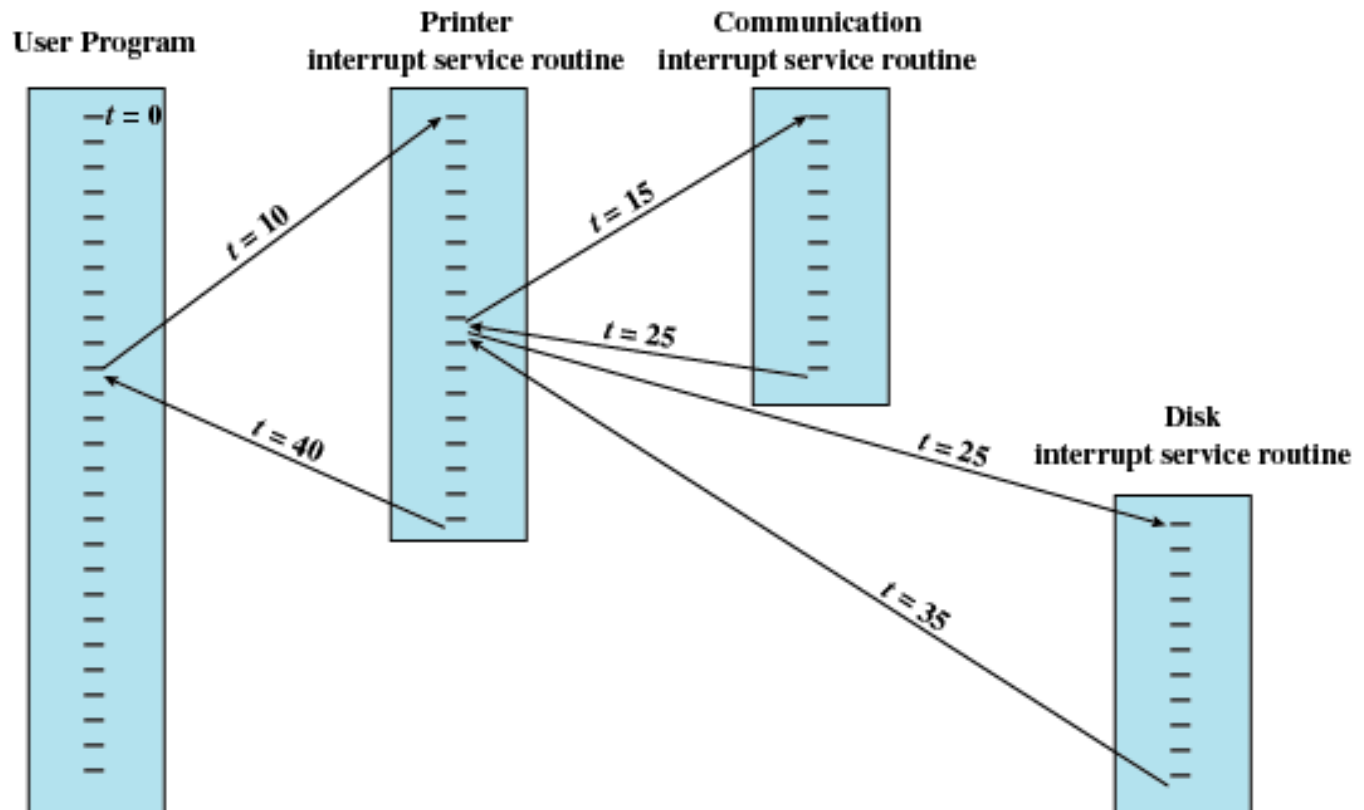


Figure 1.13 Example Time Sequence of Multiple Interrupts

Multiprogramming

- Processor has more than one program to execute
- The sequence the programs are executed depend on their relative priority and whether they are waiting for I/O
- After an interrupt handler completes, control may not return to the program that was executing at the time of the interrupt

Memory Hierarchy

- The design constraints on a computer's memory can be summed up by three questions:
 - How much?
 - How fast?
 - How expensive? (Tradeoff)
-
- Faster access time, greater cost per bit
 - Greater capacity, smaller cost per bit
 - Greater capacity, slower access speed

Memory Hierarchy

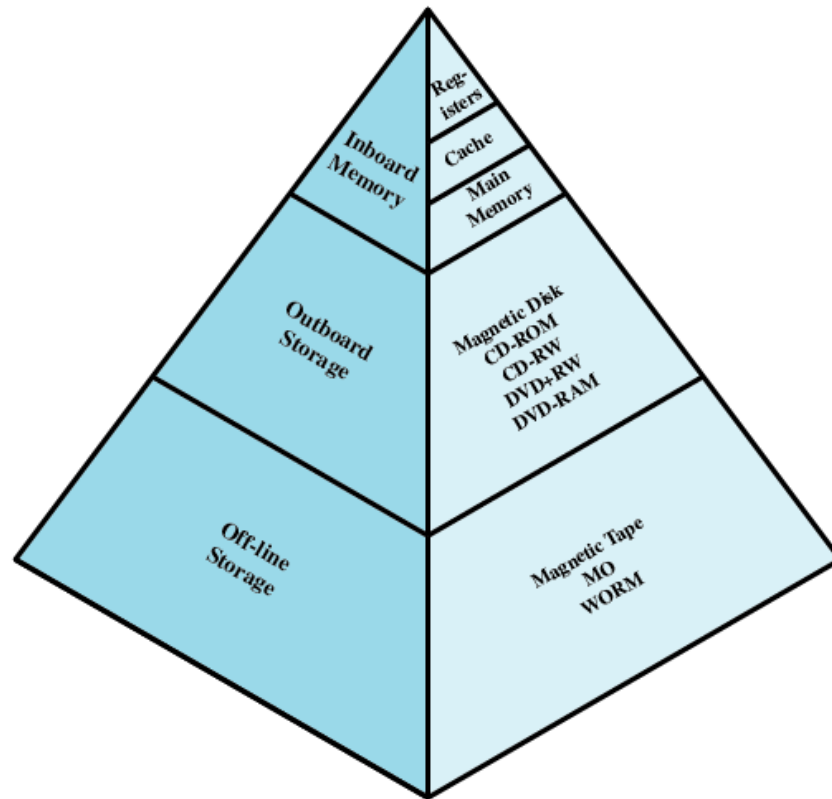


Figure 1.14 The Memory Hierarchy

Going Down the Hierarchy

- Decreasing cost per bit
- Increasing capacity
- Increasing access time
- Decreasing frequency of access of the memory by the processor
 - Locality of reference

Going Down the Hierarchy

- Locality of reference / principle of locality:
 - Tendency of the computer program to access the same set of memory locations for a particular time period.
 - Spatial Locality
 - Temporal Locality

Exercise Given:

Suppose that the processor has access to two levels of memory. Level 1 contains 1,000 bytes and has an access time of $0.1\ \mu\text{s}$; level 2 contains 100,000 bytes and has an access time of $1\ \mu\text{s}$. Assume that if a byte to be accessed is in level 1, then the processor accesses it directly. If it is in level 2, then the byte is first transferred to level 1 and then accessed by the processor. (for simplicity we ignore the transfer time)

- In our example, suppose 95% of the memory accesses are found in the cache . Then the average time to access a byte:

Exercise Given:

Memory hit / Miss:

A Memory hit refers to the situation wherein the memory is able to successfully find data and supply to the application.

Memory Hit Ratio :

A hit ratio is a calculation of memory hits, and comparing them with how many total content requests were received.

Average Time to access a byte:

$$(0.95) * (0.1 \text{ s}) + (0.05) * (0.1 \text{ s} + 1 \text{ s}) = 0.095 + 0.055 = 0.15 \text{ s}$$

Going Down the Hierarchy

Temporal Locality :

- This type of optimization includes bringing in the frequently accessed memory references to a nearby memory location for a short duration of time so that the future accesses are much faster.
- **Eg. if in an instruction set we have a variable declared that is being accessed very frequently we bring in that variable in a memory register which is the nearest in memory hierarchy for faster access.**

Going Down the Hierarchy

Spatial Locality :

- Assumes that if a memory location has been accessed it is highly likely that a nearby/consecutive memory location will be accessed as well

Hence we bring in the nearby memory references too in a nearby memory location for faster access.

E.g : Traversal of a one-dimensional array in any instruction set will benefit from this optimization.

Going Down the Hierarchy

Does the strategy of using two memory levels works in principle, but only if **conditions(a) through (d) applies.**

a)Decreasing cost per bit

b)Increasing capacity

c)Increasing access time

d)Decreasing frequency of access of the memory by the processor

Memory

- Registers, Cache, and Primary memory
 - Employs semi conductor technology
 - Volatile
 - May be visible to the programmer in terms of individual bytes or words
- Secondary / Auxiliary memory
 - Nonvolatile
 - Used to store program and data files
 - visible to the programmer only in terms of files and records,
 - A hard disk is also used to provide an extension to main memory known as **virtual memory**

Disk Cache

- A portion of main memory used as a buffer to temporarily to hold data for the disk.
- Disk writes are clustered
- Some data written out may be referenced again. The data are retrieved rapidly from the software cache instead of slowly from disk

Cache Memory

- Invisible to operating system
- But, Interacts with other memory management hardware.
- **Motivation**
- In every instruction cycle, processor access memory many times (to fetch instruction, operands and store result).
- Problem: The rate at which the processor can execute instructions is clearly limited by the **memory cycle time (Bottleneck)**
- Ideally, Technology to built processor registers, should be used to built main memory. (Expensive)

Cache Memory

- **Solution:**
- Exploit the principle of locality by providing a **small, fast memory between the processor and main memory.**
- Increase the speed of memory access
- Still with a price of less expensive types.
- Exploit the principle of locality
- There is a relatively large and slow main memory together with a smaller, faster cache memory.

Cache Memory

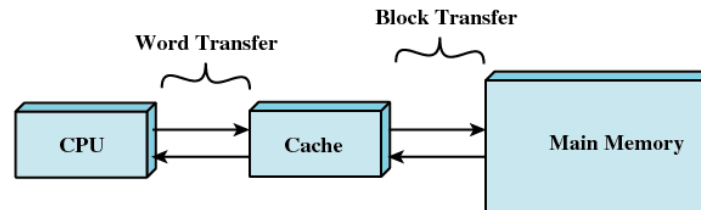
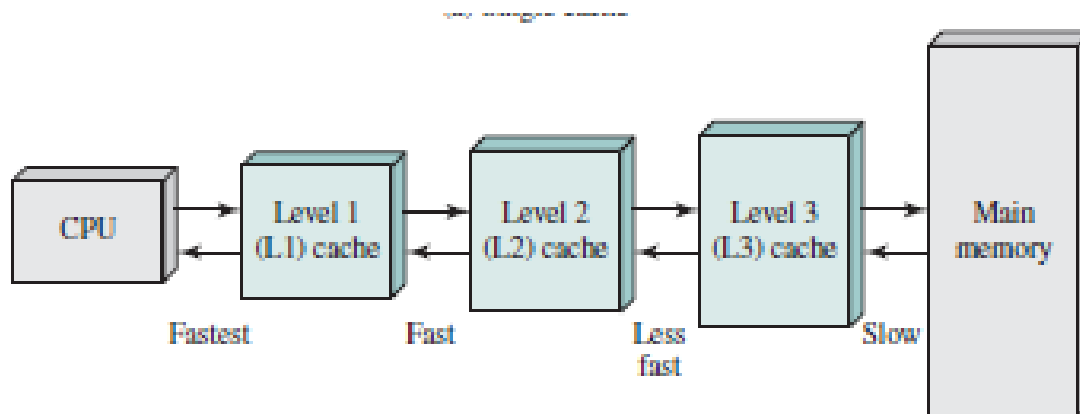


Figure 1.16 Cache and Main Memory

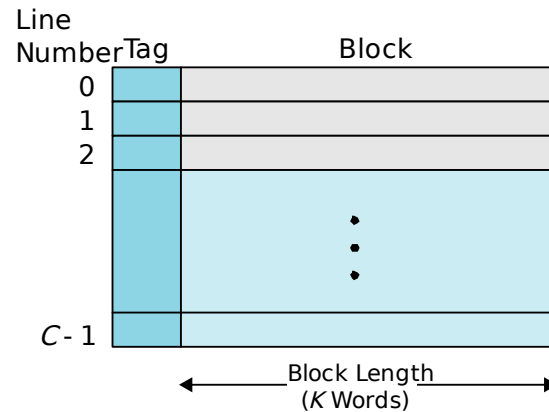


(b) Three-level cache organization

Cache Memory

- Contains a copy of a portion of main memory
- When processor wants a byte or word, cache is first checked for it.
- If not found in cache, fixed number of blocks of memory containing the needed information is moved to the cache and delivered to the processor.
- Locality of reference plays role that this process is not so frequent.

Cache/Main Memory System

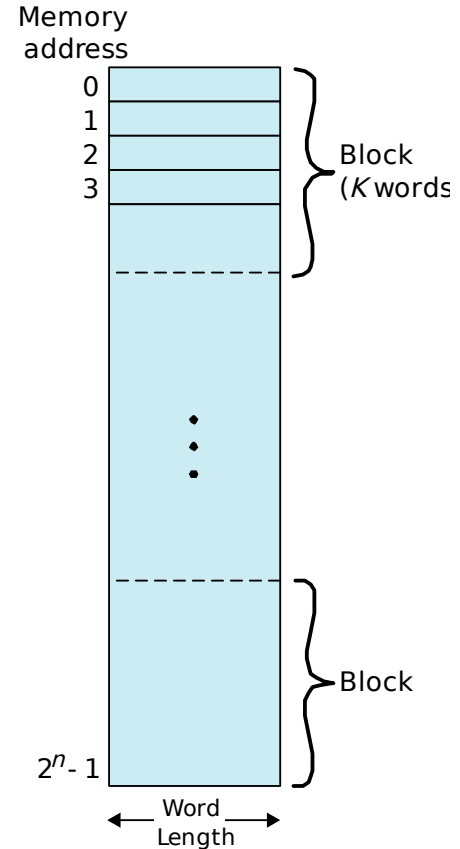


(a) Cache

$$M(\text{No. of Blocks}) = 2^n / K$$

&

$$C \ll M$$



(b) Main memory

Figure 1.17 Cache/Main-Memory Structure

Cache Read Operation

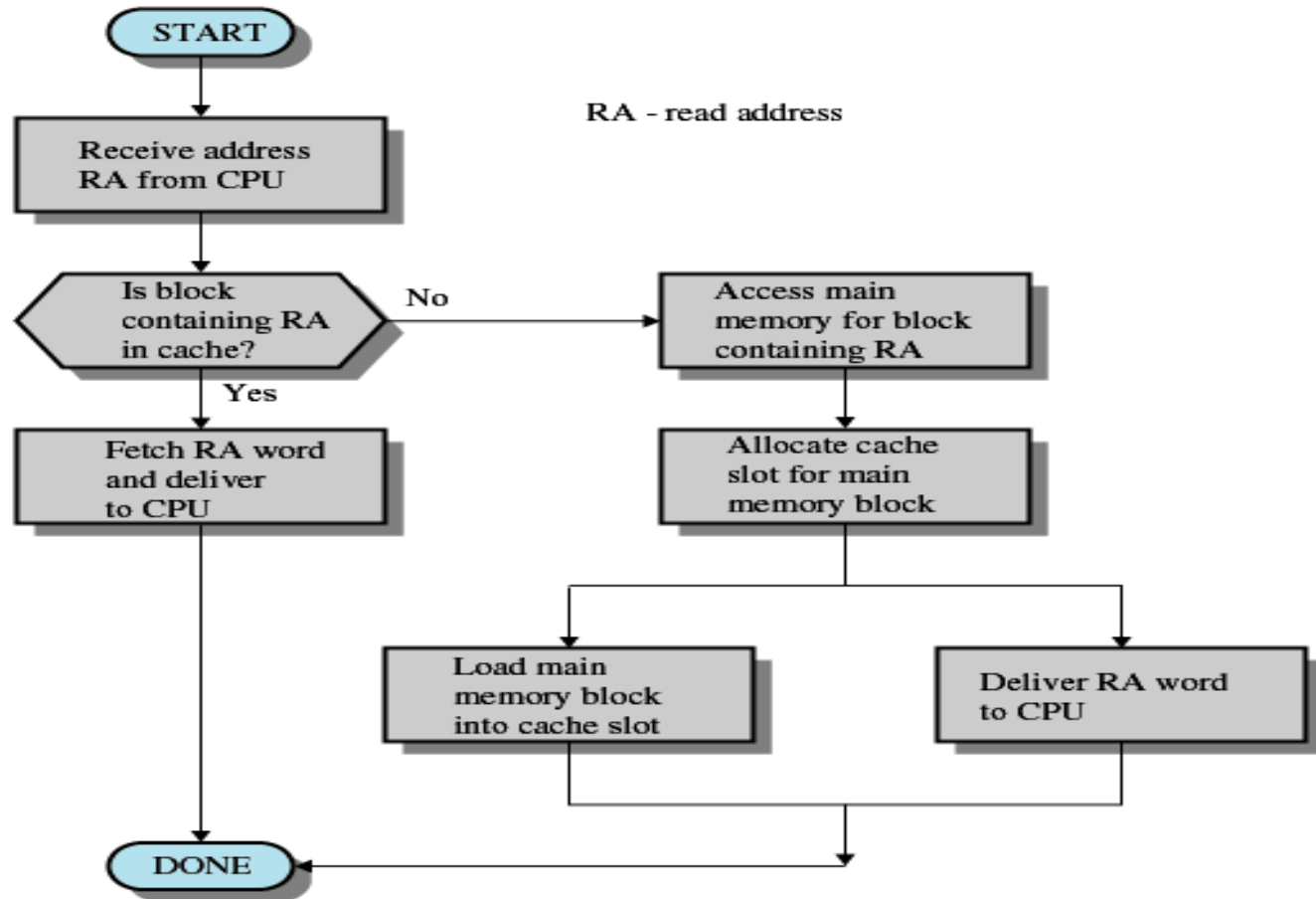


Figure 1.18 Cache Read Operation

Cache Design

- Cache size
 - Small caches have a significant impact on performance
- Block size
 - The unit of data exchanged between cache and main memory
 - Larger block size more hits until probability of using newly fetched data becomes less than the probability of reusing data that have to be moved out of cache

Cache Design

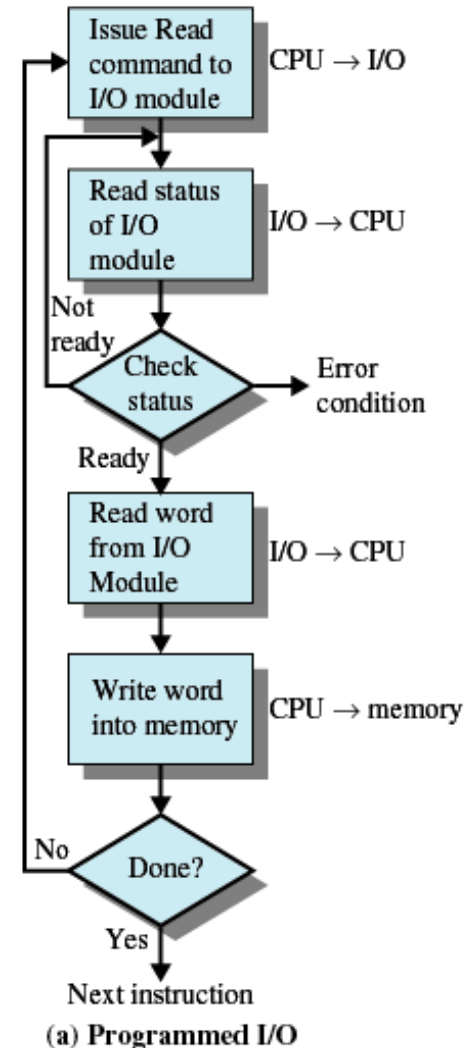
- Mapping function
 - Determines which cache location the block will occupy
 - Once cache filled (which to replace?)
 - Target – maximize hit ratio.
- Replacement algorithm
 - Determines which block to replace
 - Least-Recently-Used (LRU) algorithm

Cache Design

- Write policy
 - When the memory write operation takes place
 - Can occur every time block is updated
 - Can occur only when block is replaced
 - Minimizes memory write operations
 - Leaves main memory in an obsolete state

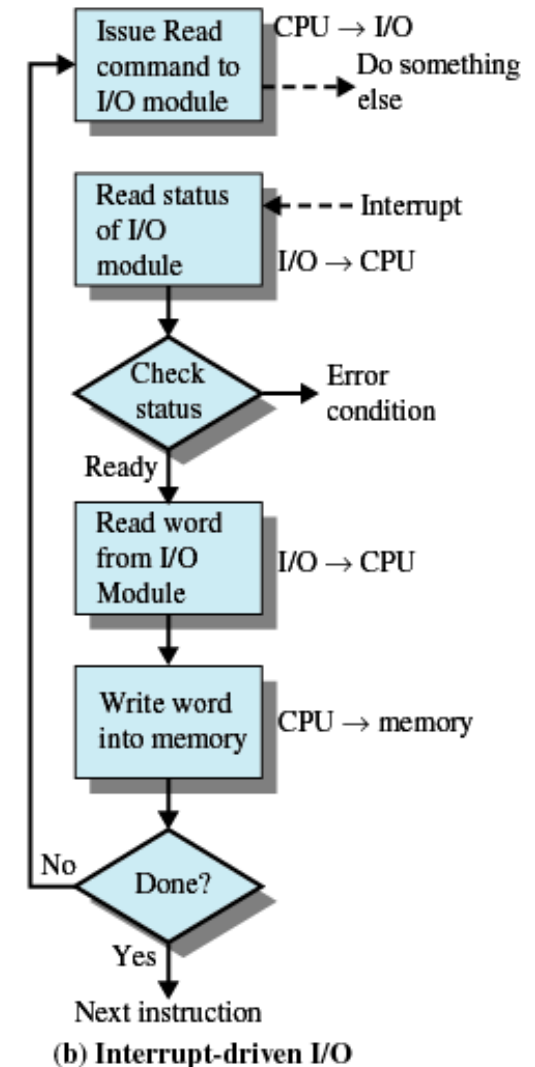
Programmed I/O

- I/O module performs the action, not the processor
- Sets appropriate bits in the I/O status register
- No interrupts occur
- Processor checks status until operation is complete



Interrupt-Driven I/O

- Processor is interrupted when I/O module ready to exchange data
- Processor saves context of program executing and begins executing interrupt-handler
- No needless waiting
- Consumes a lot of processor time because every word read or written passes through the processor

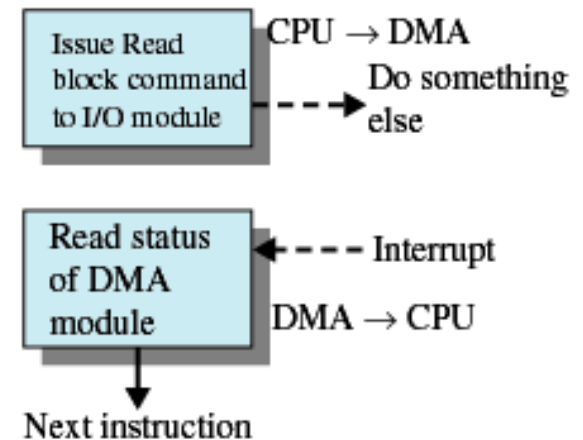


Direct Memory Access (DMA)

- I/O exchanges occur directly with memory
- Processor grants I/O module authority to read from or write to memory
- Relieves the processor responsibility for the exchange

Direct Memory Access

- Transfers a block of data directly to or from memory
- An interrupt is sent when the transfer is complete
- Processor continues with other work



(c) Direct memory access

MULTIPROCESSOR AND MULTICORE ORGANIZATION

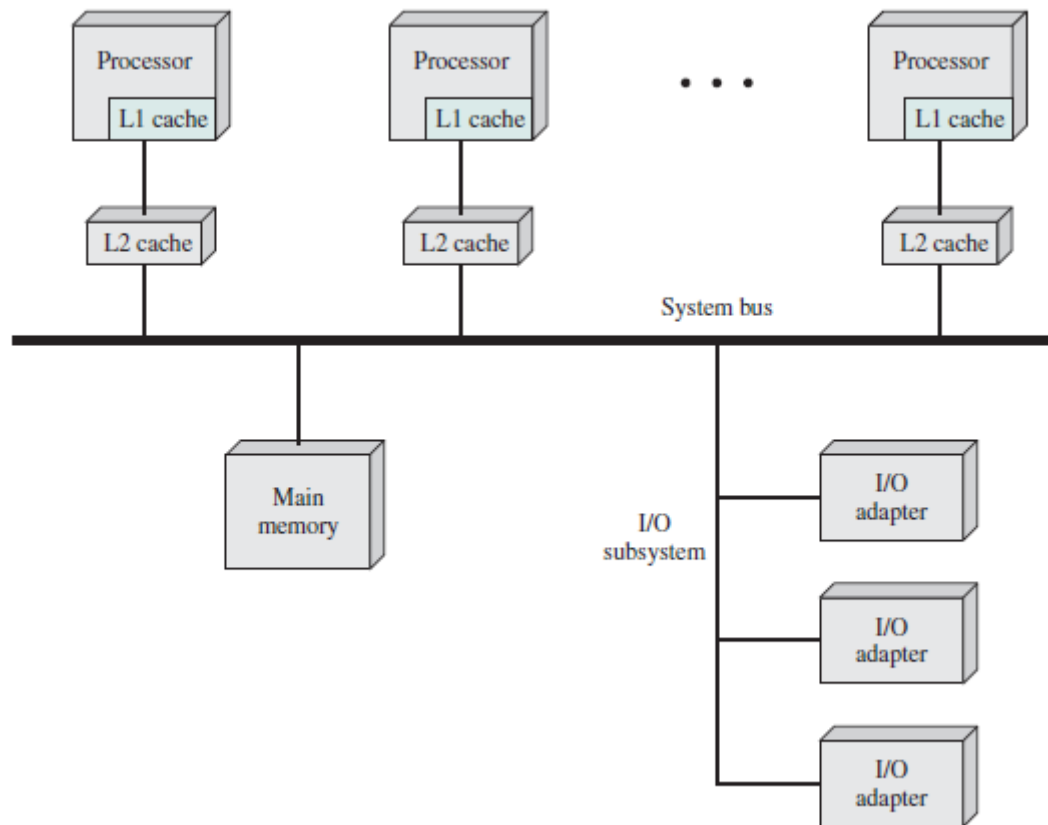
- The three most popular approaches to providing parallelism:
- How?
- by replicating processors:
 - symmetric multiprocessors (SMPs),
 - multicore computers,
 - clusters.

MULTIPROCESSOR AND MULTICORE ORGANIZATION

An SMP can be defined as a stand-alone computer system with the following characteristics:

1. There are two or more similar processors of comparable capability.
2. These processors share the same main memory and I/O facilities and are interconnected by a bus or other internal connection scheme, such that memory access time is approximately the same for each processor.
3. All processors share access to I/O devices, either through the same channels or through different channels that provide paths to the same device.
4. All processors can perform the same functions (hence the term *symmetric*).
5. The system is controlled by an integrated operating system that provides interaction between processors and their programs at the job, task, file, and data element levels.

MULTIPROCESSOR AND MULTICORE ORGANIZATION



Symmetric Multiprocessor

MULTIPROCESSOR AND MULTICORE ORGANIZATION

- Cache Coherence Problem :

Problem :

- Processors generally have at least one level of cache
- This use of cache introduces some new design considerations.
- Local cache contains an image of a portion of main memory, if a word is altered in one cache, it could invalidate a word in another cache.

Solution :

- Other processors must be alerted that an update has taken place.
- Addressed in hardware rather than by the OS. 6

MULTIPROCESSOR AND MULTICORE ORGANIZATION

- SMP Vs Uniprocessor Organization (Potential Advantages)
 - **Performance:** some portions of the work can be done in parallel (If the work to be done by a computer can be organized in such a way)
 - **Availability :** The failure of a single processor does not halt the machine. Instead, the system can continue to function at reduced performance (all processors can perform the same functions.)
 - **Incremental growth:** A user can enhance the performance of a system by adding an additional processor. Incrementally
 - **Scaling:** Vendors can offer a range of products with different price and performance characteristics based on the number of processors configured in the system.

MULTIPROCESSOR AND MULTICORE ORGANIZATION

Multicore Computers (chip multiprocessor) :

- combines two or more processors (called cores) on a single piece of silicon (called a die).
- Typically, each core consists of all of the components of an independent processor
 - Registers
 - ALU
 - pipeline hardware
 - control unit
 - L1 instruction and data caches.
 - Contemporary multicore chips also contain L2 cache and sometimes even L3.

MULTIPROCESSOR AND MULTICORE ORGANIZATION

Multicore Computers (chip multiprocessor) :

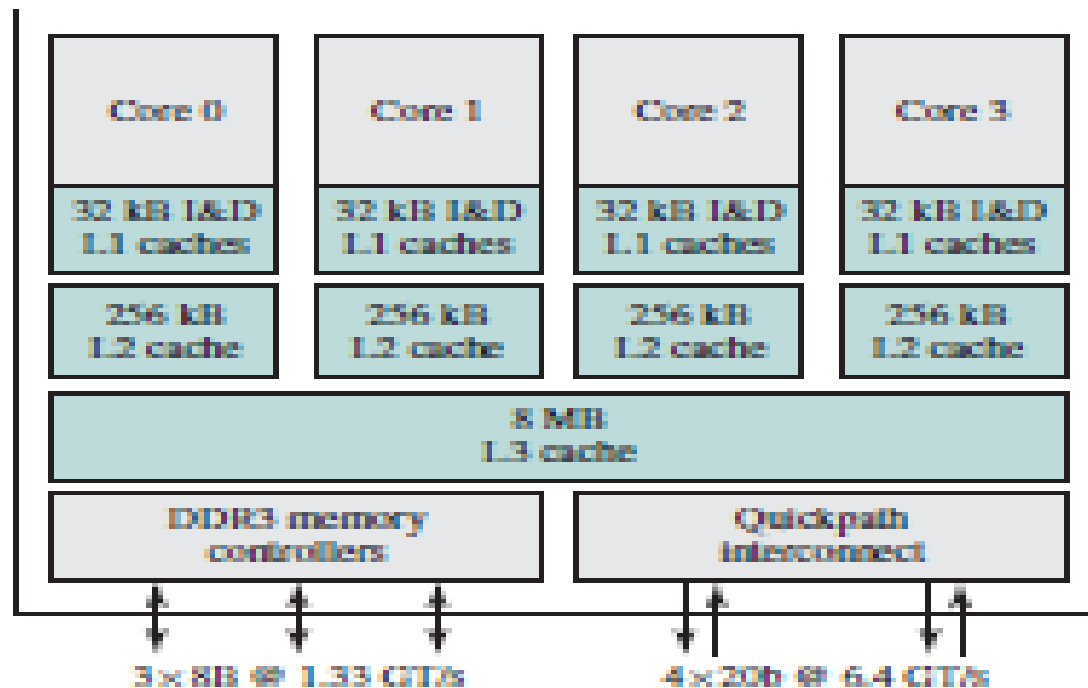


Figure 1.20 Intel Core i7 Block Diagram

Practice Exercise 1:

Consider a hypothetical 32-bit microprocessor having 32-bit instructions composed of two fields. The first byte contains the opcode and the remainder an immediate operand or an operand address.

- a.** What is the maximum directly addressable memory capacity (in bytes)?
- b.** Discuss the impact on the system speed if the Microprocessor bus has
 - 1.** a 32-bit local address bus and a 16-bit local data bus, or
 - 2.** a 16-bit local address bus and a 16-bit local data bus.
- c.** How many bits are needed for the program counter and the instruction register?

Practice Exercise 1: (Solution)

a. $2^{(32-8)} = 2^{24} = 16,777,216 \text{ bytes} = 16 \text{ MB}$

—(8 bits = 1 byte for the opcode).

b.1. a 32-bit local address bus and a 16-bit local data bus. Instruction and data transfers would take three bus cycles each, one for the address and two for the data. Since If the address bus is 32 bits, the whole address can be transferred to memory at once and decoded there; however, since the data bus is only 16 bits, it will require 2 bus cycles (accesses to memory) to fetch the 32-bit instruction or operand.

.

Practice Exercise 1: (Solution)

- **b.2.** a 16-bit local address bus and a 16-bit local data bus.
Instruction and data transfers would take four bus cycles each, two for the address and two for the data. Therefore, that will have the processor perform two transmissions in order to send to memory the whole 32-bit address; this will require more complex memory interface control to latch the two halves of the address before it performs an access to it. In addition to this two-step address issue, since the data bus is also 16 bits, the microprocessor will need 2 bus cycles to fetch the 32-bit instruction or operand.
- **c.** For the PC needs 24 bits (24-bit addresses), and for the IR needs 32 bits (32-bit addresses).

Practice Exercise 2:

Consider a memory system with the following parameters:

$T_c = 100$ ns, $C_c = 0.01$ cents/bit, $T_m = 1200$ ns, $C_m = 0.001$ cents/bit

- a. What is the cost of 1 MByte of main memory?
- b. What is the cost of 1 MByte of main memory using cache memory technology?